Day 2: Flush → ΔT 10 °C (cooling) → Dose, one START press

Prepared for Shadow · Team 13 · Lam Research Challenge 2026 · Stage 3 Day 2 · Target colour Chocolate

Contents

- Overview
- Requirement check
- What changed from Day 1
- P&ID
- Plumbing
- Electrical
- Firmware
- Temperature probe calibration
- Before pressing START
- The judged run
- Logging and the presentation
- Troubleshooting
- Assumptions and open points
- Appendix: firmware source

Overview

Day 2 turns the Chocolate plant into one automatic sequence. A single press of START runs three steps, and each starts only when the one before reports DONE.

Step	What the plant does	DONE when
1 Flush	SV4 opens. P1 pulls water from the flush tank T-4 through tee TA and FT1 and out of the red outlet into the 2 L jug, which stands inside the mixing tank. FT1 totalises.	FT1 reads 2,000 mL, P1 has switched itself off and SV4 is closed. The log says to lift the jug out
2 ΔT	SV5 opens and P4 circulates yellow from the bottom of T-2 through the copper coil in the ice bath and back in at the top. TT-1 (yellow) and TT-2 (flush water) are read every 0.8 s.	$\lvert TT\text{-}1 - TT\text{-}2 \rvert \geq 10.3$ °C on 3 readings in a row, P4 off, SV5 closed
3 Dose	Yellow 300 → Green 900 → Red 1,800 mL into T-5, each closed-loop on its own flow sensor with a slow finish.	All three dosed, all pumps off, all five valves closed

The interlock trips **every pump at once** and closes every valve on no flow, an empty tank, a full mixing tank or jug, a lost sensor link, no cooling in the loop, or START pressed during the run. After the run, or a trip, the controller prints a report block with the four things the judges want: flush run, calibration table, ΔT trend and dosed volumes. `tools/report.py` turns the saved serial log into a page with charts.

Biggest unknown: how long Step 2 takes. The tanks are full, so the loop has to cool about 10 L of yellow by 10 °C. That is about 420 kJ, or roughly 1.3 kg of ice melted. With a small coil, expect about 20 to 40 minutes; this is an estimate, not a measurement. Time one cooling run with water on Day 2 morning (`loop 1` and `tlog 1`) so you know.

Requirement check

Every line of the Day 2 sheet against what the plant and firmware do. "Met" means the code does it and the host simulation showed it; nothing here has run on the real plant yet.

Sheet says	How it is met	Status
Steps 1 → 2 → 3 as one automatic sequence from a single press of START	START switch on G14 (or `start`) runs `runStart()`. A second press in the first 3 s is ignored (switch bounce, double press); a press after that stops everything as an emergency stop.	Met
No manual action; no operator input between steps	The firmware never waits for anyone: Step 1 DONE calls Step 2, Step 2 DONE calls Step 3. **But lifting the jug out of the mixing tank during Step 2 is a manual action by a person.**	**Ask the judges**
Each step starts only when the previous step is DONE	Step 2 starts only after P1 is off, SV4 is closed, FT1 has stopped counting and the 10 s pause (`TUNE_PAUSE_S`) has run out by itself. Step 3 starts only after P4 is off and SV5 is closed.	Met
Route 2 L of flush water through any one base-colour line, using its pump and valves	Red line: flush valve SV4 → tee TA → P1 → FT1 → red outlet. SV1 stays shut so no water reaches the red tank.	Met
Discharge the flush into the 2 L water jug	Jug stands in T-5 under the red outlet. The jug count trips at 2,100 mL.	Met (check the jug holds a little over 2 L to the brim)
Totalise on that line's flow sensor; pump OFF automatically at 2.0 L	FT1 pulses / k4. Slow duty for the last 200 mL, P1 stops ~3 mL early so the coast lands on 2,000. Simulated: 1,998.5 mL.	Met once k4 is calibrated with water
Create ΔT of 10 °C between any two tanks	Yellow tank T-2 cooled by P4 through the coil in the ice bath. TT-1 in yellow, TT-2 in the flush tank T-4. Probe offsets from `tcal` take out the ±0.5 °C sensor tolerance.	Met
At ΔT ≥ 10 °C hand over to Step 3 with no operator input	Needs ≥ 10.3 °C on 3 fresh readings in a row (about 2.4 s), then P4 off, SV5 closed, Step 3.	Met
Dose all three base colours in one continuous automated run	Yellow 300 → Green 900 → Red 1,800 mL, back to back.	Met
Close the loop on each flow sensor: stop the pump and close the valve at target	Each line counts its own sensor (FT1/FT2/FT3). At target minus the learned coast the pump stops, 150 ms later its valve closes, then the final volume is read.	Met
Each base colour dosed to its target volume	Simulated errors −1.3 / −2.4 / −1.6 mL. The red line still holds flush water after Step 1, so red now meters extra by `TUNE_RED_HOLDUP_ML` (measure it, see below).	Met once hold-up is set
All pumps de-energise and every solenoid valve returns to closed	`allOff()`: all four PWM to 0, then all five valves LOW. The log prints a `SAFE STATE` line listing every pump and valve.	Met
Present logged data: flush run, calibration table, ΔT trend, dosed volumes	Report block after every run or trip, and `tools/report.py` builds the page and CSVs, including the probe calibration rows.	Met

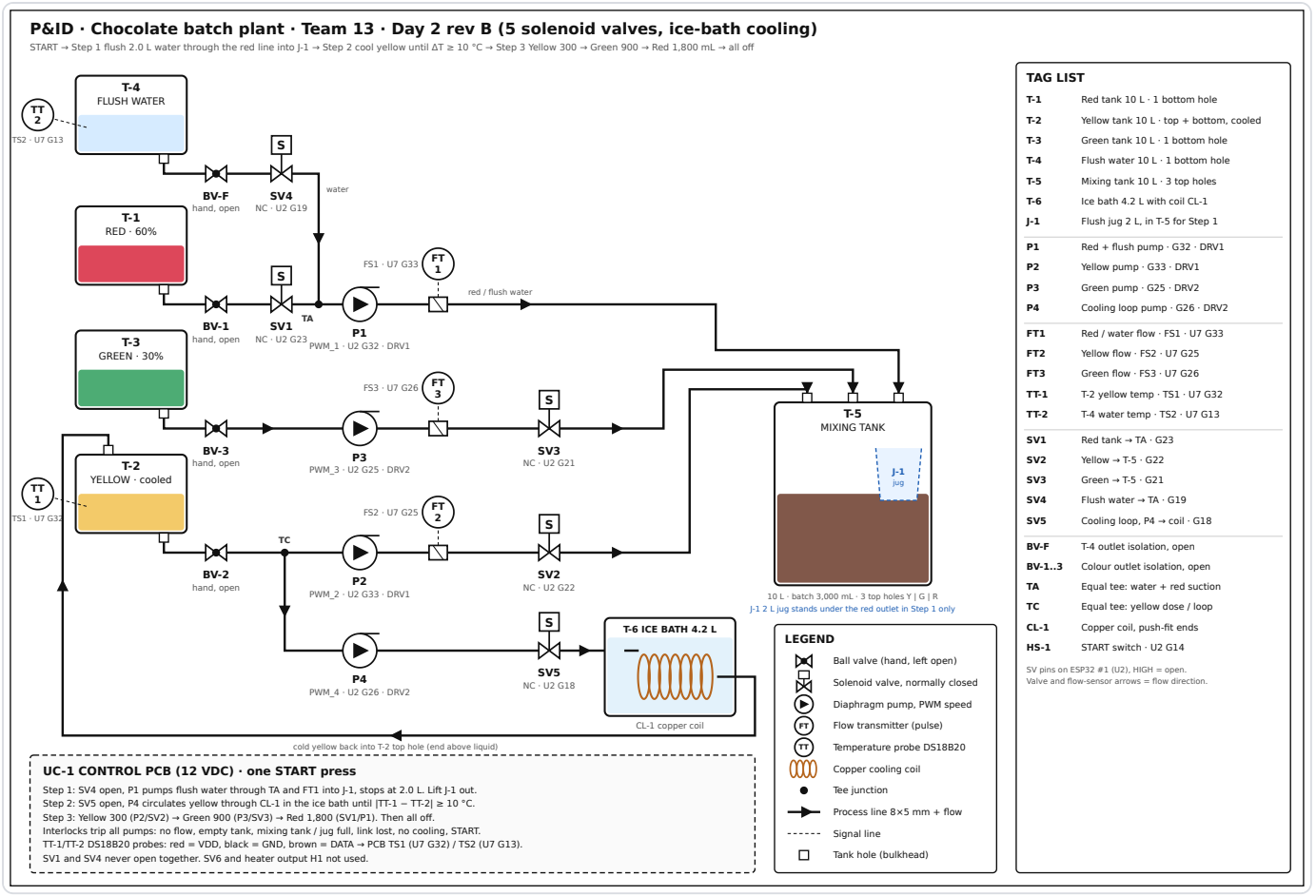
Sheet says	How it is met	Status
Interlock: trip all pumps at once on an empty tank	No float switches in the kit, so an empty tank shows as no pulses on that line's sensor: trip after 4 s at start, 2 s while running. In Step 2 the loop has no sensor, so TT-1 must fall 0.3 °C in every 3-minute window.	Met (by flow)
Interlock: no flow	Same no-pulse check on every dosing line and on manual pump runs.	Met
Interlock: full mixing tank	Volume count of T-5 (stops at 9,000 mL; START refused if the batch would not fit). FSW5 float input is ready if you fit one.	Met (by volume count)
The judges confirm the batch matches Chocolate	Depends on the base colours and dose accuracy; the hold-up fix keeps red at 60 %.	On the day

The one open point: the jug. The red outlet carries both the flush and the red dose, so the jug has to be out of T-5 before red is dosed. The firmware does not wait for it, but a person still lifts it. Ask the judges directly: "May we lift the jug out of the mixing tank during Step 2 while the sequence keeps running?" If they say no, the flush outlet needs its own valve pair after FT1 (a jug valve and a mixing-tank valve), which needs a 6th solenoid valve.

What changed from Day 1

Area	Day 1	Day 2
Solenoid valves	4, plus hand valves BV-M / BV-J	5. SV1 red source, SV2 yellow, SV3 green, SV4 flush water source, SV5 cooling loop. No hand valve moves during the run
Flush path	P4 pushed water down the header into the jug	SV4 and tee TA put flush water into the **red pump suction**, so Step 1 uses the red line's pump, flow sensor and valves. The jug stands in the mixing tank under the red outlet (allowed by the judges)
Red line	Tank → P1 → FT1 → SV1 → header	Tank → SV1 → TA → P1 → FT1 → its own top hole in T-5
Mixing tank	One header into the middle hole	Each colour has its own tube into one of the three top holes
Tanks	Two-hole tank = flush	Two-hole tank = yellow, cooled (bottom = outlet, top = loop return). A one-hole tank = flush water
P4	Flush pump	Cooling loop pump: T-2 → TC → P4 → SV5 → coil → T-2 top
Tees	TE-1, Y-1, Y-2, TE-2 in the header	TA (water + red) and TC (yellow dose + loop). Both Y tees spare
Sensors	FT1–FT3	+ two DS18B20 probes on U7 TS1 G32 / TS2 G13
Sequence	START = colour batch, 2nd START = flush	START = Flush → Cool → Dose, all automatic
Kept	k 1.8719 / 9.6364 / 1.9033 pulses/mL, fast 100 % and slow 60 % with stall boost, learned overrun, MAX_DOSE_MS 600,000, order Yellow → Green → Red, TUNING block with `code`, no comments in the code	

P&ID



Day 2 P&ID rev B. Source: `pid/pid_day2.py`; also `pid/chocolate-pid-day2.svg` and `.png` for printing.

Step	Open valves	Pump
1 Flush	SV4	P1
2 Cool	SV5	P4
3 Yellow	SV2	P2
3 Green	SV3	P3
3 Red	SV1	P1
Done / trip	none	none

The firmware never lets SV1 and SV4 open together (flush water would drain by gravity into the red tank) and only starts P1 with exactly one of them open. `Day2_Plant_Setup.html` animates each step.

Plumbing

1. **Red + flush:** T-1 bottom → BV-1 → SV1 (arrow away from the tank) → TA run port. Flush tank bottom → BV-F → SV4 (arrow away from the tank) → TA side port. TA → P1 → FT1 (arrow with the flow) → T-5 left top hole.
2. **Green:** T-3 bottom → BV-3 → P3 → FT3 → SV3 → T-5 middle top hole.
3. **Yellow (two-hole tank):** bottom hole → BV-2 → TC. TC run port → P2 → FT2 → SV2 → T-5 right top hole.
4. **Cooling loop:** TC side port → P4 → SV5 (arrow towards the coil) → coil inlet. Coil outlet → T-2 top hole, tube end above the liquid so it can't siphon. Coil fully under the ice.
5. Tube ends in T-5 stay above the liquid. The 2 L jug stands under the red outlet for Step 1 only.
6. Count: male 8 mm connectors 5 SV × 2 + 4 BV × 2 = 18 of 20. Equal tees TA and TC, both Y tees spare. The coil has its own push-fit ends.

Electrical

Item	PCB	Pin	Notes
SV1 red source	SV1	U2 G23	now on the suction side
SV2 / SV3	SV2 / SV3	U2 G22 / G21	as Day 1
SV4 flush water source	SV4	U2 G19	now on the suction side
SV5 cooling loop	SV5	U2 G18	new
P4 loop pump	DRV2 M2	PWM_4 U2 G26	as Day 1
TT-1 probe, yellow T-2	TS1	U7 G32	DS18B20: red = VDD, black = GND, brown = DATA
TT-2 probe, flush T-4	TS2	U7 G13	same. Both probes on TS1 also works: the 2nd sensor on that bus is used as TT-2

Probe wiring check. Red / brown / black is not the usual DS18B20 colour code. Before plugging in, check that the JST plug pins match the TS header labels on the PCB. A DS18B20 wired backwards gets hot in seconds: unplug it at once. Then `scan` on ESP32 #2 (U7) should report 1 sensor on each bus. The PCB is assumed to have the data pull-up resistor; if `scan` finds nothing, add 4.7 kΩ from DATA to VDD.

Step 2 time and ice

Cooling needs 4.19 kJ per litre per °C, and melting ice absorbs 334 kJ/kg. For 10.3 °C:

Yellow in T-2	Heat to remove	Ice melted (minimum)
6 L	259 kJ	0.8 kg
8 L	345 kJ	1.0 kg
10 L	431 kJ	1.3 kg

The 4.2 L bath holds about 3 kg of ice if packed, so it is enough for one run, but have spare ice. Stir the bath now and then: still water around the coil warms up and slows everything. Put TT-1 at mid-depth in T-2. Cold liquid sinks, so a probe on the bottom reaches ΔT sooner; whether that is fair is a question for the judges. Also ask whether the ice bath counts as one of "any two tanks": if so, ΔT is met almost at once.

Firmware

New PlatformIO project `firmware/chocolate_day2`: environment `sensor` for ESP32 #2 (U7), `controller` for ESP32 #1 (U2). The sensor build pulls the OneWire and DallasTemperature libraries, so the laptop needs internet for the first build.

1. USB on ESP32 #2 (U7): `pio run -e sensor -t upload`, then `pio device monitor` and type `temps`.
2. Move the cable to ESP32 #1 (U2): `pio run -e controller -t upload`. Leave it there for the run.

TUNING block (the first lines of controller main.cpp, right under the includes)

Name	Value	Meaning
TUNE_K[4]	1.8719, 9.6364, 1.9033, 1.8719	pulses/mL for Red, Yellow, Green, Water. Water uses FT1, so it starts at the red value until you calibrate it
TUNE_TARGET[4]	1800, 300, 900, 2000	mL. The 4th is the flush
TUNE_FAST / TUNE_SLOW	100 / 60 %	slow for the last 10 % (at least 20 mL), +10 boost if the pump stalls
TUNE_ORDER	2, 3, 1	Yellow → Green → Red. Yellow goes first, right after it was cooled
TUNE_PAUSE_S	10	seconds of wait between Step 1 DONE and Step 2 start, with everything off. Runs by itself; 0 = no pause
TUNE_DT_TARGET / MARGIN / CONFIRM	10.0 °C / 0.3 / 3	hand-over at ≥ 10.3 °C on 3 readings in a row
TUNE_DT_TIMEOUT_MIN	60	trip if ΔT is not reached (0 = wait forever)
TUNE_LOOP_DUTY	100	P4 duty in Step 2
TUNE_LOOP_CHECK_S / MIN_DROP	180 s / 0.3 °C	TT-1 must drop this much in every window of this length during Step 2, or all pumps trip (no circulation, blocked loop or ice used up). 0 turns it off
TUNE_RED_HOLDUP_ML	0.0	mL of flush water left in the red line (TA → P1 → FT1 → outlet) after Step 1. Red meters this much extra so the full 1,800 mL of red reaches T-5. **Set it before the run**
TUNE_TS_OFFSET[2]	0.00, 0.00	°C added to TS1 / TS2. Filled in by `tcal` then `code`
TUNE_TEMP_STALE_MS	5000	a probe may miss readings for this long before Step 2 trips (one bad reading no longer stops the run)
TUNE_START_LOCKOUT_MS	3000	START presses this soon after the start are ignored
TUNE_MIX_MAX_ML / JUG_MAX_ML	9000 / 2100	volume-count limits for the full-tank interlock
TUNE_FLOAT_*	off	for float switches on FSW1–4 / FSW5, if you ever fit them

As on Day 1, `SETTINGS_FROM_CODE = true`: code values win at every boot. Change live with `set`, then `code` and paste the printed block into main.cpp.

Commands

Board	Command	What it does
ESP32 #1 (U2)	`start` / `abort`	same as the START switch / trip everything
ESP32 #1 (U2)	`status`, `show`, `temps`	live valves, pumps, counts, temperatures / settings / temperatures only
ESP32 #1 (U2)	`prime 4 10`, `prime 1 10`	water / red through P1 for 10 s (put the jug under the red outlet)
ESP32 #1 (U2)	`calrun 4 60` then `k 4 <mL>`	water calibration of FT1 into the jug; adds a row to the calibration table
ESP32 #1 (U2)	`loop 1` / `loop 0`	cooling loop by hand (SV5 + P4)
ESP32 #1 (U2)	`tlog 1`	log temperatures every 2 s while idle, to time a cooling test
ESP32 #1 (U2)	`tcal ice` / `tcal match` / `tcal clear`	probe offset calibration (see the probe section)
ESP32 #1 (U2)	`set holdup 40`	red line hold-up in mL
ESP32 #1 (U2)	`set pause 10`	seconds between Step 1 and Step 2
ESP32 #1 (U2)	`cal`, `report`	calibration table / stored report of the last run
ESP32 #1 (U2)	`jug 0`, `mix 0`	reset the volume counts after emptying
ESP32 #1 (U2)	`valve 1..5 0|1`, `pump 1..4 %`	manual, idle only, interlocked
ESP32 #1 (U2)	`set dt 0`	test mode: skips Step 2 until the next reboot
ESP32 #2 (U7)	`show`, `watch`, `temps`, `scan`	pulse counts / live rates and temps / temps / re-find probes and print their ROM code and family (0x28 = DS18B20)

Temperature probe calibration

The probes have no model number, so first check what they are: `scan` on ESP32 #2 (U7) prints each probe's ROM code and family byte. **0x28 = DS18B20** (also what most clones report), 0x10 = DS18S20, 0x22 = DS1822. A DS18B20 is factory-calibrated to ±0.5 °C, so it needs no gain calibration, but two probes can disagree by up to 1 °C. Step 2 is judged on the difference between two probes, so take that offset out.

Ice point (best): `tcal ice`

1. Fill a cup with crushed ice and a little water, stir, wait 2 minutes. That is 0.0 °C.
2. Put both probe tips in the middle of the cup, not touching the sides. Keep stirring.
3. On ESP32 #1 (U2) type `temps` until both are steady (about 30 s), then `tcal ice`. It averages 8 readings, refuses if they are still moving more than 0.25 °C or are not within 2 °C of zero, and prints a `Q` row per probe.
4. Type `code`, paste the `TUNE_TS_OFFSET` line into the top of main.cpp and upload again, so the offsets survive a reboot.

Quick version: `tcal match`

Both probes in the same stirred water at room temperature, then `tcal match`: TS2 is shifted to read the same as TS1. This fixes ΔT (what is judged) but not the absolute temperature. `tcal clear` resets both offsets.

The offsets are printed in the calibration table (`O` rows) and in the report, so you can show the judges that ΔT was measured with matched probes.

Before pressing START

Do these in order on Day 2 morning, with the serial monitor logging. Each command says which board it runs on.

1. **Flash** ESP32 #2 (U7) with `pio run -e sensor -t upload`, then ESP32 #1 (U2) with `pio run -e controller -t upload`. Leave the USB cable on U2.
2. **Probes:** on ESP32 #2 (U7) type `scan`. Both TS1 and TS2 should show family 0x28 and CRC ok. If one is missing, swap brown and red on that plug (red VDD, black GND, brown DATA is assumed).
3. **Probe calibration:** `tcal ice` on ESP32 #1 (U2), as in the section above, then `code` and paste TUNE_TS_OFFSET.
4. **Valves and interlock:** on ESP32 #1 (U2) `valve 1 1` then `valve 4 1` must answer `! interlock: SV1 and SV4 may never be open together`. Click-test SV2, SV3, SV5, then `off`.
5. **Prime** every line into the jug or a cup on ESP32 #1 (U2): `prime 4 10` (water), `prime 1 10` (red), `prime 2 5`, `prime 3 5` (hold a cup under the yellow and green outlets so nothing lands in T-5). Then `loop 1` until liquid comes back out of the coil into T-2, then `loop 0`. Finish with `prime 1 5` so the red line is full of red.
6. **Water calibration of FT1** on ESP32 #1 (U2): empty jug, `calrun 4 60`, weigh the water (1 g = 1 mL), `k 4 <mL>`. A 1.5 % k error moved the simulated flush by 30 mL. Redo a short `calrun` + `k` on lines 1 to 3 too, so the calibration table has Day 2 rows.
7. **Red hold-up:** measure the tube from tee TA through P1 and FT1 to the end of the red outlet. Hold-up ≈ 19.6 mL per metre of 8×5 tube + about 10 mL for P1 and FT1. Type `set holdup <mL>` on ESP32 #1 (U2).
8. **Cooling test** (if there is time): ice in the bath, `tlog 1` and `loop 1` on ESP32 #1 (U2). TT-1 must fall well over 0.3 °C in 3 minutes. Note the °C per minute, then `loop 0`, `tlog 0`.
9. **Interlock test with water:** during `prime 4 20` shut BV-F: P1 trips within about 2 s. Unplug TT-2 and type `start`: refused.
10. **Lock in the settings:** `code` on ESP32 #1 (U2), paste the whole printed block into the TUNING block, upload again.
11. **Set up the judged run:** empty jug in T-5 under the red outlet, T-5 otherwise empty, fresh ice packed around the coil, all ball valves open, base tanks stirred, TT-1 at mid-depth in yellow away from the return stream, TT-2 below the water line of T-4 (it drops 2 L in Step 1).
12. **Final check** on ESP32 #1 (U2): reboot, then `status`: link OK, all valves 0, both temperatures read, no last trip. `mix 0` if T-5 was emptied since boot.
13. Start `pio device monitor` so the log is saved, and press START once.

The judged run

1. Everything in "Before pressing START" done. Press START once.
2. Step 1 ends with `LIFT THE JUG OUT OF THE MIXING TANK NOW`. The sequence goes straight on into Step 2 and does not wait. Only lift the jug if the judges have agreed to it (see Requirement check); red is dosed last, after the whole of Step 2.

```
#RUN,1
> dose WATER 2000.0 mL (into the JUG (jug must sit under the red outlet))
= WATER target 2000.0 mL, metered 1998.5 mL ...
> STEP 1 DONE: 1998.5 mL flushed to the jug, P1 off, SV4 closed
>>> LIFT THE JUG OUT OF THE MIXING TANK NOW <<<
> pause 10 s, Step 2 starts by itself
> STEP 2: SV5 open, P4 100% circulating YELLOW through the ice-bath coil until |TS1 - TS2| >= 10.3 C
~ cooling check 1: TS1 dropped 1.59 C in the last 180 s (need 0.30)
~ cooling check 2: TS1 dropped 1.48 C in the last 180 s (need 0.30)
> STEP 2 DONE: dT = 10.32 C (TS1 13.98, TS2 24.30) after 1455.2 s, P4 off, SV5 closed
> dose YELLOW 300.0 mL (into the MIXING TANK)
...
SAFE STATE: P1=0% P2=0% P3=0% P4=0% SV1=closed SV2=closed SV3=closed SV4=closed SV5=closed
```

These numbers are from the host simulation, not the real plant; the cooling rate in particular is made up. With a hold-up set, red prints `RED line still holds ~40.0 mL of flush water ...` and meters target + hold-up; the report shows the red that reached T-5.

Logging and the presentation

`platformio.ini` has `monitor_filters = direct, log2file`, so every `pio device monitor` session is saved as `platformio-device-monitor-*.log` in the project folder.

Tag	Meaning
`D,t,line,phase,duty,pulses,mL,mL/s`	flow trace, 4 per second (line 4 = flush water)
`T,t,TS1,TS2,dT,P4`	temperature trend every 2 s through the run; last column 1 while the cooling pump runs
`R,line,name,target,metered,error,pulses,s`	result of the flush and each dose
`K,…` / `C,…`	k in use / calibration runs
`O,TSn,offset` / `Q,n,TSn,method,raw,ref,offset`	probe offsets in use / probe calibrations done this session
`H,1,RED,holdup,metered`	red line hold-up added to the red dose
`S,t,event` / `!,TRIP,t,why`	step changes / trips

If the laptop log is lost, type `report` on ESP32 #1 (U2) and save the output as a text file.

```
python3 tools/report.py platformio-device-monitor-XXXX.log
```

This writes `report.html` (flush chart, calibration table, ΔT trend, dosed volumes, sequence log) plus CSV files. Python 3 only, no packages. `tools/example_SIMULATED` shows the output from a simulated run; don't present those numbers. To get real calibration rows, redo a short `calrun` + `k` on each line on Day 2 morning with the monitor logging.

Troubleshooting

Message	Cause and fix
`temperature sensors TS1/TS2 not both reading, run refused`	Probe unplugged or wired wrong. `scan` on U7. For a flush/dose test without probes use `set dt 0`.
`TRIP no circulation in the cooling loop`	P4 not pumping (air in the loop, SV5 not opening, BV-2 shut) or no ice. Prime the loop with `loop 1` first. Raise `set loopcheck` if cooling is just slow.
`TRIP cooling stopped`	TT-1 stopped falling later in Step 2: ice used up, loop blocked or air-locked. Add ice and stir the bath; if cooling is just slow near the end, `set loopdrop 0.15`.
`! need 8 good readings` / `! readings still moving`	Wait until both probes are steady in the stirred ice water and try again.
`TRIP dT not reached in time`	Not enough ice or the bath water warmed up. More ice, stir the bath, `set dttimeout 90`.
`TRIP … WATER: no flow after start`	Flush tank empty, BV-F shut, SV4 not opening.
`TRIP jug full`	Jug count over 2,100 mL. Usually k4 too low: calibrate water.
`TRIP lost sensor link`	U7 reset or UART wiring. `status` shows link OK/DOWN.
`slow duty stalled the pump, raised to 70%`	Normal for Yellow, more likely when cold. `code` after the run keeps the new value.

Assumptions and open points

- **Probe wiring:** red = VDD, black = GND, brown = DATA is the likely mapping, not confirmed. `scan` on U7 proves it: family 0x28 with CRC ok means the probe is wired right.
- **Which tanks:** TT-1 in the cooled yellow tank, TT-2 in the flush-water tank. Any two tanks satisfy the rule; move TT-2 if a judge prefers.
- **Shared tee TC:** idle diaphragm pumps block backflow, so the loop and the dosing line share TC without another valve. If P2 pulls air through the loop, close BV-2 briefly and re-prime with `prime 2 5` (tube end in the jug).
- **No flow sensor on the loop:** circulation is proven by TT-1 falling.
- **Cold yellow:** slightly thicker, so FT2's k may shift a little. Recalibrate yellow with chilled liquid if there is time.
- **Jug in T-5:** the judges allowed placing it there for the flush. Lifting it out during Step 2 is a manual action between steps; confirm that with them too.
- **Build:** `pio run` passes on Shadow's laptop. Logic was also checked with a host compiler and a plant simulation: normal run, empty red tank, START as stop, double START press, missing probe, one bad probe reading, dead cooling loop, ice running out, Step 2 skipped, red hold-up, probe calibration.

Appendix: firmware source

platformio.ini

```
[platformio]
default_envs = controller

[env]
platform = espressif32@6.9.0
board = nodemcu-32s
framework = arduino
upload_port = /dev/ttyUSB0
monitor_port = /dev/ttyUSB0
upload_speed = 460800
monitor_speed = 115200
monitor_filters = direct, log2file
build_flags = -Wall

[env:controller]
build_src_filter = +<controller/>

[env:sensor]
build_src_filter = +<sensor/>
lib_deps =
    paulstoffregen/OneWire@2.3.8
    milesburton/DallasTemperature@3.11.0
```

include/link.h

```
#pragma once
#include <Arduino.h>

static const uint32_t LINK_BAUD      = 115200;
static const uint8_t  LINK_RX_PIN    = 16;
static const uint8_t  LINK_TX_PIN    = 17;
static const uint32_t LINK_PERIOD_MS = 10;
static const int32_t  TEMP_INVALID   = -12700;

inline uint8_t linkChecksum(const char *s, size_t n) {
  uint8_t x = 0;
  for (size_t i = 0; i < n; i++) x ^= (uint8_t)s[i];
  return x;
}

inline int linkSeal(char *buf, size_t cap, int n) {
  uint8_t x = linkChecksum(buf, n);
  return n + snprintf(buf + n, cap - n, "%02X\n", x);
}

inline bool linkCheck(const char *buf) {
```

```
  const char *star = strchr(buf, '*');
  if (!star) return false;
  return (uint8_t)strtoul(star + 1, nullptr, 16) == linkChecksum(buf, star - buf + 1);
}
```

src/sensor/main.cpp (ESP32 #2, U7)

```
#include <Arduino.h>
#include <OneWire.h>
#include <DallasTemperature.h>
#include "link.h"

static const uint8_t FS_PIN[3] = {33, 25, 26};
static const uint8_t TS_PIN[2] = {32, 13};
static const uint8_t FSW_PIN[2] = {34, 35};
static const uint8_t H1_PIN = 23;
static const uint32_t TEMP_CONV_MS = 800;
static const uint8_t TS_RESOLUTION = 12;

static volatile uint32_t pulses[3] = {0, 0, 0};
static volatile uint32_t lastEdgeUs[3] = {0, 0, 0};
static const uint32_t MIN_EDGE_US = 300;

static void IRAM_ATTR countEdge(uint8_t i) {
  uint32_t now = micros();
  if (now - lastEdgeUs[i] >= MIN_EDGE_US) {
    pulses[i]++;
    lastEdgeUs[i] = now;
  }
}
static void IRAM_ATTR isr0() { countEdge(0); }
static void IRAM_ATTR isr1() { countEdge(1); }
static void IRAM_ATTR isr2() { countEdge(2); }

static portMUX_TYPE mux = portMUX_INITIALIZER_UNLOCKED;
static void snapshot(uint32_t out[3]) {
  portENTER_CRITICAL(&mux);
  for (int i = 0; i < 3; i++) out[i] = pulses[i];
  portEXIT_CRITICAL(&mux);
}

static OneWire bus1(TS_PIN[0]);
static OneWire bus2(TS_PIN[1]);
static DallasTemperature ds1(&bus1);
static DallasTemperature ds2(&bus2);
static int found1 = 0, found2 = 0;
static DeviceAddress addr[2];
static bool have[2] = {false, false};
static DallasTemperature *busOf[2] = {&ds1, &ds2};
static float tempC[2] = {NAN, NAN};
static uint32_t tempSeq = 0;
```

```
static uint32_t seq = 0;
static bool watch = false;

static bool validT(float t) { return !isnan(t) && t > -55.0f && t < 125.0f && t != 85.0f; }

static void scanSensors() {
  ds1.begin();
  ds2.begin();
  found1 = ds1.getDeviceCount();
  found2 = ds2.getDeviceCount();
  have[0] = found1 > 0 && ds1.getAddress(addr[0], 0);
  busOf[0] = &ds1;
  if (found2 > 0) {
    have[1] = ds2.getAddress(addr[1], 0);
    busOf[1] = &ds2;
  } else {
    have[1] = found1 > 1 && ds1.getAddress(addr[1], 1);
    busOf[1] = &ds1;
  }
  ds1.setResolution(TS_RESOLUTION);
  ds2.setResolution(TS_RESOLUTION);
  ds1.setWaitForConversion(false);
  ds2.setWaitForConversion(false);
  ds1.requestTemperatures();
  ds2.requestTemperatures();
}

static const char *familyName(uint8_t f) {
  switch (f) {
    case 0x28: return "DS18B20";
    case 0x10: return "DS18S20";
    case 0x22: return "DS1822";
    case 0x3B: return "MAX31850";
    default: return "unknown";
  }
}

static void printProbes() {
  for (int i = 0; i < 2; i++) {
    Serial.printf("TS%d: ", i + 1);
    if (!have[i]) {
      Serial.println(F("no sensor found (check red VDD, black GND, brown DATA and the 4.7k pull-up)"
      continue;
    }
    for (int b = 0; b < 8; b++) Serial.printf("%02X", addr[i][b]);
    bool crcOk = OneWire::crc8(addr[i], 7) == addr[i][7];
    Serial.printf("  family 0x%02X = %s  CRC %s  bus %s  parasite %s\n", addr[i][0], familyName(addr
                  crcOk ? "ok" : "BAD", busOf[i] == &ds1 ? "G32" : "G13", busOf[i]->isParasitePowerM
  }
}

static void readTemps() {
  float a = have[0] ? busOf[0]->getTempC(addr[0]) : NAN;
```

```
  float b = have[1] ? busOf[1]->getTempC(addr[1]) : NAN;
  tempC[0] = validT(a) ? a : NAN;
  tempC[1] = validT(b) ? b : NAN;
  tempSeq++;
  ds1.requestTemperatures();
  ds2.requestTemperatures();
}

static void sendTemps() {
  char buf[80];
  long a = validT(tempC[0]) ? lroundf(tempC[0] * 100) : TEMP_INVALID;
  long b = validT(tempC[1]) ? lroundf(tempC[1] * 100) : TEMP_INVALID;
  int n = snprintf(buf, sizeof(buf), "T,%lu,%ld,%ld,0*", (unsigned long)tempSeq, a, b);
  linkSeal(buf, sizeof(buf), n);
  Serial2.print(buf);
}

static void printTemps() {
  Serial.printf("TS1 (G32, %d found) %s  TS2 (G13, %d found) %s  dT %s\n", found1,
                validT(tempC[0]) ? String(tempC[0], 2).c_str() : "--", found2,
                validT(tempC[1]) ? String(tempC[1], 2).c_str() : "--",
                validT(tempC[0]) && validT(tempC[1]) ? String(fabsf(tempC[0] - tempC[1]), 2).c_str()
}

void setup() {
  pinMode(H1_PIN, OUTPUT);
  digitalWrite(H1_PIN, LOW);
  Serial.begin(115200);
  Serial2.begin(LINK_BAUD, SERIAL_8N1, LINK_RX_PIN, LINK_TX_PIN);
  for (int i = 0; i < 3; i++) pinMode(FS_PIN[i], INPUT_PULLUP);
  for (int i = 0; i < 2; i++) pinMode(FSW_PIN[i], INPUT);
  attachInterrupt(digitalPinToInterrupt(FS_PIN[0]), isr0, FALLING);
  attachInterrupt(digitalPinToInterrupt(FS_PIN[1]), isr1, FALLING);
  attachInterrupt(digitalPinToInterrupt(FS_PIN[2]), isr2, FALLING);
  scanSensors();
  Serial.printf("\n[U7 sensor, Day 2] FS1=G33 FS2=G25 FS3=G26  TS1=G32 (%d on bus)  TS2=G13 (%d on b
  printProbes();
  Serial.println(F("Commands: show, watch, temps, scan"));
}

void loop() {
  static uint32_t nextFrame = 0, nextWatch = 0, nextTemp = TEMP_CONV_MS;
  static uint32_t prevWatch[3] = {0, 0, 0};
  uint32_t now = millis();

  if ((int32_t)(now - nextTemp) >= 0) {
    nextTemp = now + TEMP_CONV_MS;
    readTemps();
    sendTemps();
  }

  if ((int32_t)(now - nextFrame) >= 0) {
    nextFrame = now + LINK_PERIOD_MS;
    uint32_t c[3];
```

```
    snapshot(c);
    int fsw = (digitalRead(FSW_PIN[0]) ? 1 : 0) | (digitalRead(FSW_PIN[1]) ? 2 : 0);
    char buf[80];
    int n = snprintf(buf, sizeof(buf), "F,%lu,%lu,%lu,%lu,%d*", (unsigned long)seq++,
                     (unsigned long)c[0], (unsigned long)c[1], (unsigned long)c[2], fsw);
    linkSeal(buf, sizeof(buf), n);
    Serial2.print(buf);
  }

  if (watch && (int32_t)(now - nextWatch) >= 0) {
    nextWatch = now + 500;
    uint32_t c[3];
    snapshot(c);
    Serial.printf("FS1 %lu (%lu Hz)  FS2 %lu (%lu Hz)  FS3 %lu (%lu Hz)  ",
                  (unsigned long)c[0], (unsigned long)((c[0] - prevWatch[0]) * 2),
                  (unsigned long)c[1], (unsigned long)((c[1] - prevWatch[1]) * 2),
                  (unsigned long)c[2], (unsigned long)((c[2] - prevWatch[2]) * 2));
    printTemps();
    for (int i = 0; i < 3; i++) prevWatch[i] = c[i];
  }

  static String rx;
  bool gotLine = false;
  while (Serial.available() && !gotLine) {
    char ch = Serial.read();
    if (ch == '\n' || ch == '\r') gotLine = rx.length() > 0;
    else if (rx.length() < 40) rx += ch;
  }
  if (gotLine) {
    String cmd = rx;
    rx = "";
    cmd.trim();
    if (cmd == "show") {
      uint32_t c[3];
      snapshot(c);
      Serial.printf("FS1 %lu  FS2 %lu  FS3 %lu  FSW5 %d  FSW6 %d\n", (unsigned long)c[0], (unsigned
                    (unsigned long)c[2], digitalRead(FSW_PIN[0]), digitalRead(FSW_PIN[1]));
    } else if (cmd == "watch") {
      watch = !watch;
      Serial.printf("watch %s\n", watch ? "on" : "off");
    } else if (cmd == "temps") {
      printTemps();
    } else if (cmd == "scan") {
      scanSensors();
      Serial.printf("TS1 bus: %d sensor(s), TS2 bus: %d sensor(s)\n", found1, found2);
      printProbes();
    } else if (cmd.length()) {
      Serial.println(F("commands: show, watch, temps, scan"));
    }
  }
}
```

src/controller/main.cpp (ESP32 #1, U2)

```
#include <Arduino.h>
#include <Preferences.h>
#include "link.h"

static const bool SETTINGS_FROM_CODE = true;

static const float TUNE_K[4] = {1.8719f, 9.6364f, 1.9033f, 1.8719f};
static const float TUNE_TARGET[4] = {1800.0f, 300.0f, 900.0f, 2000.0f};
static const uint8_t TUNE_FAST[4] = {100, 100, 100, 100};
static const uint8_t TUNE_SLOW[4] = {60, 60, 60, 60};
static const float TUNE_OVERRUN[4] = {3.0f, 3.0f, 3.0f, 3.0f};
static const uint8_t TUNE_ORDER[3] = {2, 3, 1};
static const float TUNE_BAND_PCT = 10.0f;
static const float TUNE_BAND_MIN = 20.0f;
static const bool TUNE_LEARN = true;
static const float TUNE_RED_HOLDUP_ML = 0.0f;

static const uint32_t TUNE_PAUSE_S = 10;
static const float TUNE_DT_TARGET = 10.0f;
static const float TUNE_DT_MARGIN = 0.3f;
static const uint8_t TUNE_DT_CONFIRM = 3;
static const uint32_t TUNE_DT_TIMEOUT_MIN = 60;
static const uint8_t TUNE_LOOP_DUTY = 100;
static const uint32_t TUNE_LOOP_CHECK_S = 180;
static const float TUNE_LOOP_MIN_DROP = 0.3f;
static const uint32_t TUNE_TLOG_MS = 2000;
static const float TUNE_TS_OFFSET[2] = {0.00f, 0.00f};
static const uint32_t TUNE_TEMP_STALE_MS = 5000;

static const float TUNE_MIX_MAX_ML = 9000.0f;
static const float TUNE_JUG_MAX_ML = 2100.0f;
static const bool TUNE_FLOAT_LOW[4] = {false, false, false, false};
static const bool TUNE_FLOAT_MIX_FULL = false;
static const uint8_t TUNE_FLOAT_ALARM_LEVEL = LOW;
static const uint32_t TUNE_START_LOCKOUT_MS = 3000;

static const uint8_t SV_PIN[5] = {23, 22, 21, 19, 18};
static const uint8_t PWM_PIN[4] = {32, 33, 25, 26};
static const uint8_t FSW_PIN[4] = {36, 39, 34, 35};
static const uint8_t START_PIN = 14;

static const uint32_t PWM_FREQ = 20000;
static const uint8_t PWM_BITS = 10;

enum { SV_RED_SRC = 0, SV_YEL = 1, SV_GRN = 2, SV_WATER_SRC = 3, SV_LOOP = 4, NSV = 5 };
enum { L_RED = 0, L_YELLOW = 1, L_GREEN = 2, L_WATER = 3, NLINES = 4 };
enum { P_RED = 0, P_YEL = 1, P_GRN = 2, P_LOOP = 3 };

struct LineDef
{
  const char *name;
```

```
  uint8_t pump;
  uint8_t sensor;
  int8_t valveA;
  int8_t valveB;
};
static const LineDef LD[NLINES] = {
    {"RED", P_RED, 0, SV_RED_SRC, -1},
    {"YELLOW", P_YEL, 1, SV_YEL, -1},
    {"GREEN", P_GRN, 2, SV_GRN, -1},
    {"WATER", P_RED, 0, SV_WATER_SRC, -1},
};


struct LineCfg
{
  float k;
  float target;
  uint8_t fast;
  uint8_t slow;
  float overrun;
};
struct Config
{
  LineCfg line[NLINES];
  uint8_t order[3];
  float bandPct;
  float bandMinMl;
  bool learn;
  float dtTarget;
  float dtMargin;
  uint8_t dtConfirm;
  uint32_t dtTimeoutMin;
  uint8_t loopDuty;
  uint32_t loopCheckS;
  float loopMinDrop;
  float mixMaxMl;
  float jugMaxMl;
  float tsOffset[2];
  float redHoldupMl;
  uint32_t pauseS;
};
static Config cfg;
static Preferences prefs;

static void setDefaults()
{
  for (int i = 0; i < NLINES; i++)
    cfg.line[i] = {TUNE_K[i], TUNE_TARGET[i], TUNE_FAST[i], TUNE_SLOW[i], TUNE_OVERRUN[i]};
  for (int i = 0; i < 3; i++)
    cfg.order[i] = TUNE_ORDER[i] - 1;
  cfg.bandPct = TUNE_BAND_PCT;
  cfg.bandMinMl = TUNE_BAND_MIN;
  cfg.learn = TUNE_LEARN;
  cfg.dtTarget = TUNE_DT_TARGET;
```

```
  cfg.dtMargin = TUNE_DT_MARGIN;
  cfg.dtConfirm = TUNE_DT_CONFIRM;
  cfg.dtTimeoutMin = TUNE_DT_TIMEOUT_MIN;
  cfg.loopDuty = TUNE_LOOP_DUTY;
  cfg.loopCheckS = TUNE_LOOP_CHECK_S;
  cfg.loopMinDrop = TUNE_LOOP_MIN_DROP;
  cfg.mixMaxMl = TUNE_MIX_MAX_ML;
  cfg.jugMaxMl = TUNE_JUG_MAX_ML;
  cfg.tsOffset[0] = TUNE_TS_OFFSET[0];
  cfg.tsOffset[1] = TUNE_TS_OFFSET[1];
  cfg.redHoldupMl = TUNE_RED_HOLDUP_ML;
  cfg.pauseS = TUNE_PAUSE_S;
}

static void printCode()
{
  const LineCfg *L = cfg.line;
  Serial.println(F("---- TUNING values (lines RED, YELLOW, GREEN, WATER) ----"));
  Serial.printf("static const float TUNE_K[4] = {%.4ff, %.4ff, %.4ff, %.4ff};\n", L[0].k, L[1].k, L[
  Serial.printf("static const float TUNE_TARGET[4] = {%.1ff, %.1ff, %.1ff, %.1ff};\n", L[0].target,
  Serial.printf("static const uint8_t TUNE_FAST[4] = {%u, %u, %u, %u};\n", L[0].fast, L[1].fast, L[2
  Serial.printf("static const uint8_t TUNE_SLOW[4] = {%u, %u, %u, %u};\n", L[0].slow, L[1].slow, L[2
  Serial.printf("static const float TUNE_OVERRUN[4] = {%.1ff, %.1ff, %.1ff, %.1ff};\n", L[0].overrun
  Serial.printf("static const uint8_t TUNE_ORDER[3] = {%d, %d, %d};\n", cfg.order[0] + 1, cfg.order[
  Serial.printf("static const float TUNE_BAND_PCT = %.1ff;\n", cfg.bandPct);
  Serial.printf("static const float TUNE_BAND_MIN = %.1ff;\n", cfg.bandMinMl);
  Serial.printf("static const bool TUNE_LEARN = %s;\n", cfg.learn ? "true" : "false");
  Serial.printf("static const float TUNE_RED_HOLDUP_ML = %.1ff;\n", cfg.redHoldupMl);
  Serial.printf("static const uint32_t TUNE_PAUSE_S = %lu;\n", (unsigned long)cfg.pauseS);
  Serial.printf("static const float TUNE_DT_TARGET = %.1ff;\n", cfg.dtTarget);
  Serial.printf("static const float TUNE_DT_MARGIN = %.1ff;\n", cfg.dtMargin);
  Serial.printf("static const uint8_t TUNE_DT_CONFIRM = %u;\n", cfg.dtConfirm);
  Serial.printf("static const uint32_t TUNE_DT_TIMEOUT_MIN = %lu;\n", (unsigned long)cfg.dtTimeoutMi
  Serial.printf("static const uint8_t TUNE_LOOP_DUTY = %u;\n", cfg.loopDuty);
  Serial.printf("static const uint32_t TUNE_LOOP_CHECK_S = %lu;\n", (unsigned long)cfg.loopCheckS);
  Serial.printf("static const float TUNE_LOOP_MIN_DROP = %.1ff;\n", cfg.loopMinDrop);
  Serial.printf("static const float TUNE_MIX_MAX_ML = %.1ff;\n", cfg.mixMaxMl);
  Serial.printf("static const float TUNE_JUG_MAX_ML = %.1ff;\n", cfg.jugMaxMl);
  Serial.printf("static const float TUNE_TS_OFFSET[2] = {%.2ff, %.2ff};\n", cfg.tsOffset[0], cfg.tsO
}
static void saveConfig()
{
  prefs.begin("choc2", false);
  prefs.putBytes("cfg", &cfg, sizeof(cfg));
  prefs.end();
  Serial.println(SETTINGS_FROM_CODE
                   ? F("saved to flash, but SETTINGS_FROM_CODE is true, so the code values win at
                   : F("saved"));
}
static void loadConfig()
{
  if (SETTINGS_FROM_CODE)
  {
    setDefaults();
```

```
    return;
  }
  prefs.begin("choc2", true);
  size_t n = prefs.getBytesLength("cfg");
  if (n == sizeof(cfg))
    prefs.getBytes("cfg", &cfg, sizeof(cfg));
  else
    setDefaults();
  prefs.end();
}

static bool valveOpen[NSV] = {false};
static uint8_t pumpPct[4] = {0};
static uint32_t pumpOnMs[4] = {0};

static void writePump(int p, uint8_t pct)
{
  if (pct && !pumpPct[p])
    pumpOnMs[p] = millis();
  pumpPct[p] = pct;
  ledcWrite(p, (uint32_t)pct * ((1u << PWM_BITS) - 1) / 100);
}
static void writeValve(int v, bool open)
{
  digitalWrite(SV_PIN[v], open ? HIGH : LOW);
  valveOpen[v] = open;
}
static void pumpsOff()
{
  for (int p = 0; p < 4; p++)
    writePump(p, 0);
}
static void allOff()
{
  pumpsOff();
  delay(100);
  for (int v = 0; v < NSV; v++)
    writeValve(v, false);
}
static bool anyPump()
{
  for (int p = 0; p < 4; p++)
    if (pumpPct[p])
      return true;
  return false;
}

static bool pathOk(int p)
{
  if (p == P_RED)
    return valveOpen[SV_RED_SRC] != valveOpen[SV_WATER_SRC];
  if (p == P_YEL)
    return valveOpen[SV_YEL];
  if (p == P_GRN)
```

```
    return valveOpen[SV_GRN];
  return valveOpen[SV_LOOP];
}
static int pumpOfValve(int v)
{
  if (v == SV_YEL)
    return P_YEL;
  if (v == SV_GRN)
    return P_GRN;
  if (v == SV_LOOP)
    return P_LOOP;
  return P_RED;
}

static bool setPump(int p, uint8_t pct)
{
  if (pct > 100)
    pct = 100;
  if (pct > 0 && !pathOk(p))
  {
    if (p == P_RED)
      Serial.println(F("! interlock: P1 needs exactly one of SV1 (red) / SV4 (water) open"));
    else
      Serial.printf("! interlock: P%d needs SV%d open\n", p + 1, p == P_LOOP ? 5 : p + 1);
    return false;
  }
  writePump(p, pct);
  return true;
}
static bool setValve(int v, bool open)
{
  if (open && ((v == SV_RED_SRC && valveOpen[SV_WATER_SRC]) || (v == SV_WATER_SRC && valveOpen[SV_RE
  {
    Serial.println(F("! interlock: SV1 and SV4 may never be open together (water would run into the
    return false;
  }
  int p = pumpOfValve(v);
  if (!open && pumpPct[p])
  {
    writePump(p, 0);
    delay(100);
  }
  writeValve(v, open);
  return true;
}

static uint32_t counts[3] = {0, 0, 0};
static uint32_t lastCountChangeMs[3] = {0, 0, 0};
static uint8_t fswU7 = 3;
static uint32_t lastFrameMs = 0, framesOk = 0, framesBad = 0;
static bool linkOk() { return framesOk > 0 && millis() - lastFrameMs < 150; }

static float tempC[2] = {NAN, NAN};
static float tempRaw[2] = {NAN, NAN};
```

```
static const int TAVG = 8;
static float rawHist[2][TAVG];
static int nHist = 0, iHist = 0;
static uint32_t lastTempMs = 0, tempFrames = 0, badTempFrames = 0;
static uint32_t lastValidMs[2] = {0, 0};
static bool probeOk(int i) { return lastValidMs[i] && millis() - lastValidMs[i] < TUNE_TEMP_STALE_MS
static bool tempsOk() { return tempFrames > 0 && probeOk(0) && probeOk(1); }
static float deltaT() { return fabsf(tempC[0] - tempC[1]); }

static void pollLink()
{
  static char buf[96];
  static uint8_t n = 0;
  while (Serial2.available())
  {
    char c = Serial2.read();
    if (c == '\n')
    {
      buf[n] = 0;
      unsigned long seq, a, b, d;
      int f, h;
      long t1, t2;
      if (buf[0] == 'F' && linkCheck(buf) && sscanf(buf, "F,%lu,%lu,%lu,%lu,%d", &seq, &a, &b, &d, &
      {
        uint32_t nc[3] = {(uint32_t)a, (uint32_t)b, (uint32_t)d};
        uint32_t now = millis();
        for (int i = 0; i < 3; i++)
        {
          if (nc[i] != counts[i])
            lastCountChangeMs[i] = now;
          counts[i] = nc[i];
        }
        fswU7 = (uint8_t)f;
        lastFrameMs = now;
        framesOk++;
      }
      else if (buf[0] == 'T' && linkCheck(buf) && sscanf(buf, "T,%lu,%ld,%ld,%d", &seq, &t1, &t2, &h
      {
        tempRaw[0] = t1 <= TEMP_INVALID ? NAN : t1 / 100.0f;
        tempRaw[1] = t2 <= TEMP_INVALID ? NAN : t2 / 100.0f;
        uint32_t nowT = millis();
        for (int i = 0; i < 2; i++)
        {
          if (isnan(tempRaw[i]))
          {
            badTempFrames++;
            continue;
          }
          tempC[i] = tempRaw[i] + cfg.tsOffset[i];
          lastValidMs[i] = nowT;
        }
        rawHist[0][iHist] = tempRaw[0];
        rawHist[1][iHist] = tempRaw[1];
        iHist = (iHist + 1) % TAVG;
```

```
        if (nHist < TAVG)
          nHist++;
        lastTempMs = nowT;
        tempFrames++;
      }
      else
      {
        framesBad++;
      }
      n = 0;
    }
    else if (c != '\r')
    {
      if (n < sizeof(buf) - 1)
        buf[n++] = c;
      else
        n = 0;
    }
  }
}

struct TPoint
{
  float t, a, b;
  uint8_t h;
};
struct DPoint
{
  float t;
  uint8_t line, duty;
  float ml, rate;
};
struct Result
{
  bool done;
  float target, ml, secs;
  uint32_t pulses;
};
struct TCalRow
{
  uint8_t probe;
  char method;
  float raw, ref, offset;
};
static TCalRow qlog[8];
static int nQ = 0;

struct CalRow
{
  uint8_t line;
  uint32_t pulses, ms;
  float ml, k;
};
static const int MAX_T = 900, MAX_D = 2400, MAX_C = 16;
```

```
static TPoint tlog[MAX_T];
static DPoint dlog[MAX_D];
static CalRow clog[MAX_C];
static int nT = 0, nD = 0, nC = 0;
static uint32_t tStride = 1, tTick = 0;
static Result res[NLINES];
static float dtReached = NAN, dtSecs = 0;
static uint32_t runNo = 0, runT0 = 0;
static bool runLogging = false;
static bool tlogIdle = false;
static float runSecs() { return (millis() - runT0) / 1000.0f; }

static void logTemp(bool force)
{
  static uint32_t next = 0;
  uint32_t now = millis();
  if (!force && (int32_t)(now - next) < 0)
    return;
  next = now + TUNE_TLOG_MS;
  if (!runLogging && !tlogIdle)
    return;
  float t = runLogging ? runSecs() : now / 1000.0f;
  uint8_t loopOn = pumpPct[P_LOOP] ? 1 : 0;
  Serial.printf("T,%.1f,%.2f,%.2f,%.2f,%d\n", t, tempC[0], tempC[1], deltaT(), loopOn);
  if (!runLogging)
    return;
  if (tTick++ % tStride)
    return;
  if (nT >= MAX_T)
  {
    for (int i = 0; i < MAX_T / 2; i++)
      tlog[i] = tlog[i * 2];
    nT = MAX_T / 2;
    tStride *= 2;
  }
  tlog[nT++] = {t, tempC[0], tempC[1], loopOn};
}

enum Phase
{
  D_IDLE,
  D_OPEN,
  D_FAST,
  D_SLOW,
  D_STOP,
  D_SETTLE,
  D_DONE,
  D_FAULT
};
static const char *PHASE_NAME[] = {"idle", "open", "fast", "slow", "stop", "settle", "done", "FAULT"

static const uint32_t VALVE_MS = 200;
static const uint32_t STOP_TO_CLOSE_MS = 150;
static const uint32_t SETTLE_MS = 800;
```

```
static const uint32_t START_STALL_MS = 4000;
static const uint32_t STALL_MS = 2000;
static const uint32_t MAX_DOSE_MS = 600000;
static const uint32_t SLOW_BOOST_MS = 700;

struct Dose
{
  Phase ph = D_IDLE;
  int line = 0;
  float target = 0;
  bool timed = false;
  uint32_t runMs = 0;
  uint8_t duty = 0;
  bool toJug = false;
  uint32_t t0 = 0, tPh = 0, startCount = 0, lastPulseMs = 0, lastCount = 0;
  float mlAtStop = 0, mlFinal = 0;
  uint32_t pulsesFinal = 0;
  char fault[72] = "";
} dose;

static uint32_t lastPulses[NLINES] = {0, 0, 0, 0};
static uint32_t lastRunMs[NLINES] = {0, 0, 0, 0};
static float jugMl = 0, mixMl = 0;

static int sensorOf(int line) { return LD[line].sensor; }
static int pumpOf(int line) { return LD[line].pump; }
static float mlOf(int line, uint32_t pulses) { return cfg.line[line].k > 0 ? pulses / cfg.line[line]
static uint32_t dosePulses() { return counts[sensorOf(dose.line)] - dose.startCount; }
static bool doseActive() { return dose.ph != D_IDLE && dose.ph != D_DONE && dose.ph != D_FAULT; }

enum Seq
{
  S_IDLE,
  S_FLUSH,
  S_PAUSE,
  S_DT,
  S_DOSE,
  S_DONE,
  S_FAULT
};
static const char *SEQ_NAME[] = {"idle", "STEP 1 flush", "pause before STEP 2", "STEP 2 dT (cooling
static Seq seq = S_IDLE;
static char tripWhy[80] = "";

static void printReport();

static void trip(const char *why)
{
  pumpsOff();
  delay(100);
  for (int v = 0; v < NSV; v++)
    writeValve(v, false);
  if (doseActive())
  {
```

```
    dose.ph = D_FAULT;
    strncpy(dose.fault, why, sizeof(dose.fault) - 1);
  }
  strncpy(tripWhy, why, sizeof(tripWhy) - 1);
  bool wasRun = seq != S_IDLE && seq != S_DONE && seq != S_FAULT;
  if (wasRun)
    seq = S_FAULT;
  Serial.printf("!,TRIP,%.1f,%s\n", runLogging ? runSecs() : millis() / 1000.0f, why);
  Serial.printf("! INTERLOCK TRIP: %s. All pumps off, all valves closed.\n", why);
  if (wasRun)
  {
    Serial.printf("S,%.1f,FAULT,%s\n", runSecs(), why);
    runLogging = false;
    printReport();
  }
}

static bool loopStart()
{
  for (int v = 0; v < NSV; v++)
    if (valveOpen[v] && v != SV_LOOP)
      setValve(v, false);
  setValve(SV_LOOP, true);
  delay(VALVE_MS);
  return setPump(P_LOOP, cfg.loopDuty);
}
static void loopStop()
{
  writePump(P_LOOP, 0);
  delay(STOP_TO_CLOSE_MS);
  setValve(SV_LOOP, false);
}

static bool doseBegin(int line, float target, bool toJug, bool timed, uint32_t runMs, uint8_t duty)
{
  pollLink();
  if (!linkOk())
  {
    Serial.println(F("! no data from sensor board U7, check UART wiring and that U7 is powered"));
    return false;
  }
  if (!timed && cfg.line[line].k <= 0)
  {
    Serial.printf("! %s not calibrated (k=0). Run calrun/k first.\n", LD[line].name);
    return false;
  }
  if (toJug && (jugMl + (timed ? 0 : target) > cfg.jugMaxMl))
  {
    Serial.printf("! jug would overflow (now ~%.0f mL). Empty it and type: jug 0\n", jugMl);
    return false;
  }
  if (!toJug && !timed && mixMl + target > cfg.mixMaxMl)
  {
    Serial.printf("! mixing tank would pass %.0f mL (now ~%.0f mL). Empty it and type: mix 0\n", cfg
```

```
    return false;
  }
  if (pumpPct[P_LOOP])
    loopStop();
  for (int v = 0; v < NSV; v++)
    if (valveOpen[v])
      setValve(v, false);
  dose = Dose();
  dose.line = line;
  dose.target = target;
  dose.timed = timed;
  dose.runMs = runMs;
  dose.duty = duty;
  dose.toJug = toJug;
  setValve(LD[line].valveA, true);
  dose.startCount = dose.lastCount = counts[sensorOf(line)];
  dose.t0 = dose.tPh = dose.lastPulseMs = millis();
  dose.ph = D_OPEN;
  const char *where = toJug ? "into the JUG (jug must sit under the red outlet)" : "into the MIXING
  if (timed)
    Serial.printf("> run %s %lu ms at %u%% (%s)\n", LD[line].name, (unsigned long)runMs, duty, where
  else
    Serial.printf("> dose %s %.1f mL (%s)\n", LD[line].name, target, where);
  return true;
}

static void doseTelemetry(uint32_t now)
{
  static uint32_t nextT = 0, prevCount = 0, prevMs = 0, nextStore = 0;
  if (dose.ph < D_FAST || dose.ph > D_SETTLE || (int32_t)(now - nextT) < 0)
    return;
  nextT = now + 250;
  uint32_t p = dosePulses();
  if (p < prevCount || now - prevMs > 1000)
    prevMs = 0;
  float rate = (prevMs && now > prevMs) ? mlOf(dose.line, p - prevCount) * 1000.0f / (now - prevMs)
  prevCount = p;
  prevMs = now;
  float ml = mlOf(dose.line, p);
  float t = runLogging ? runSecs() : (now - dose.t0) / 1000.0f;
  Serial.printf("D,%.2f,%d,%s,%u,%lu,%.1f,%.1f\n", t, dose.line + 1, PHASE_NAME[dose.ph],
                pumpPct[pumpOf(dose.line)], (unsigned long)p, ml, rate);
  if (runLogging && (int32_t)(now - nextStore) >= 0 && nD < MAX_D)
  {
    nextStore = now + 500;
    dlog[nD++] = {t, (uint8_t)(dose.line + 1), pumpPct[pumpOf(dose.line)], ml, rate};
  }
}

static void doseStep()
{
  if (!doseActive())
    return;
  uint32_t now = millis();
```

```
  LineCfg &L = cfg.line[dose.line];
  int pump = pumpOf(dose.line);
  uint32_t c = counts[sensorOf(dose.line)];
  if (c != dose.lastCount)
  {
    dose.lastCount = c;
    dose.lastPulseMs = now;
  }
  uint32_t p = dosePulses();
  float ml = mlOf(dose.line, p);

  if (dose.ph <= D_SLOW && !linkOk())
  {
    trip("lost sensor link");
    return;
  }
  if (now - dose.t0 > MAX_DOSE_MS)
  {
    trip("dose took too long");
    return;
  }
  if (!dose.timed && dose.toJug && jugMl + ml > cfg.jugMaxMl)
  {
    trip("jug full");
    return;
  }
  if (!dose.timed && !dose.toJug && mixMl + ml > cfg.mixMaxMl)
  {
    trip("mixing tank full (volume count)");
    return;
  }
  if (dose.ph == D_FAST || dose.ph == D_SLOW)
  {
    uint32_t limit = (p == 0) ? START_STALL_MS : STALL_MS;
    if (dose.ph == D_SLOW && now - dose.lastPulseMs > SLOW_BOOST_MS && pumpPct[pump] < L.fast)
    {
      int up = pumpPct[pump] + 10;
      uint8_t d = up > L.fast ? L.fast : (uint8_t)up;
      writePump(pump, d);
      L.slow = d;
      dose.lastPulseMs = now;
      Serial.printf("~ %s slow duty stalled the pump, raised to %u%% (now slow%d = %u; type code to
                    LD[dose.line].name, d, dose.line + 1, d);
    }
    else if (now - dose.lastPulseMs > limit)
    {
      char why[72];
      snprintf(why, sizeof(why), "%s: %s", LD[dose.line].name,
               p == 0 ? "no flow after start (empty tank, air, valve shut?)" : "flow stopped (tank e
      trip(why);
      return;
    }
  }
  doseTelemetry(now);
```

```
float band = max(dose.target * cfg.bandPct / 100.0f, cfg.bandMinMl);
float stopAt = dose.target - L.overrun;

switch (dose.ph)
{
case D_OPEN:
  if (now - dose.tPh >= VALVE_MS)
  {
    uint8_t d = dose.timed ? dose.duty : L.fast;
    if (!dose.timed && ml >= dose.target - band)
      d = L.slow;
    if (!setPump(pump, d))
    {
      trip("interlock refused pump start");
      return;
    }
    dose.ph = (d == L.slow && !dose.timed) ? D_SLOW : D_FAST;
    dose.tPh = now;
    dose.lastPulseMs = now;
  }
  break;
case D_FAST:
  if (dose.timed)
  {
    if (now - dose.tPh >= dose.runMs)
      goto stop;
  }
  else if (ml >= stopAt)
  {
    goto stop;
  }
  else if (ml >= dose.target - band)
  {
    writePump(pump, L.slow);
    dose.ph = D_SLOW;
    dose.tPh = now;
  }
  break;
case D_SLOW:
  if (ml >= stopAt)
    goto stop;
  break;
case D_STOP:
  if (now - dose.tPh >= STOP_TO_CLOSE_MS)
  {
    setValve(LD[dose.line].valveA, false);
    dose.ph = D_SETTLE;
    dose.tPh = now;
  }
  break;
case D_SETTLE:
  if (now - dose.tPh >= SETTLE_MS && now - dose.lastPulseMs >= 300)
  {
```

```
      dose.pulsesFinal = p;
      dose.mlFinal = ml;
      lastPulses[dose.line] = p;
      lastRunMs[dose.line] = dose.timed ? dose.runMs : now - dose.t0;
      if (dose.toJug)
        jugMl += ml;
      else
        mixMl += ml;
      if (dose.timed)
      {
        Serial.printf("= calrun %s: %lu pulses in %lu ms (%.1f Hz). Now weigh the collected liquid a
                      LD[dose.line].name, (unsigned long)p, (unsigned long)dose.runMs,
                      p * 1000.0f / dose.runMs, dose.line + 1);
      }
      else
      {
        float coast = dose.mlFinal - dose.mlAtStop;
        if (cfg.learn && coast >= 0 && coast < 50)
          L.overrun = 0.5f * L.overrun + 0.5f * coast;
        Serial.printf("= %s target %.1f mL, metered %.1f mL (error %+.1f mL, %+.2f %%), coast %.1f m
                      LD[dose.line].name, dose.target, dose.mlFinal, dose.mlFinal - dose.target,
                      (dose.mlFinal - dose.target) * 100.0f / dose.target, coast, L.overrun, (unsign
                      (now - dose.t0) / 1000.0f);
      }
      dose.ph = D_DONE;
    }
    break;
  default:
    break;
  }
  return;
stop:
  writePump(pump, 0);
  dose.mlAtStop = ml;
  dose.ph = D_STOP;
  dose.tPh = now;
}

static bool floatAlarm(uint8_t level) { return level == TUNE_FLOAT_ALARM_LEVEL; }

static void interlocks()
{
  uint32_t now = millis();
  if (anyPump() && !linkOk())
  {
    trip("lost sensor link while a pump was running");
    return;
  }
  for (int i = 0; i < 4; i++)
  {
    if (!TUNE_FLOAT_LOW[i] || !floatAlarm(digitalRead(FSW_PIN[i])))
      continue;
    bool feeding = (i == 0 && pumpPct[P_RED] && valveOpen[SV_RED_SRC]) || (i == 1 && (pumpPct[P_YEL]
                   (i == 2 && pumpPct[P_GRN]) || (i == 3 && pumpPct[P_RED] && valveOpen[SV_WATER_SRC
```

```
    if (feeding)
    {
      char why[48];
      snprintf(why, sizeof(why), "low level float FSW%d (%s tank empty)", i + 1, LD[i].name);
      trip(why);
      return;
    }
  }
  if (TUNE_FLOAT_MIX_FULL && floatAlarm((fswU7 & 1) ? HIGH : LOW) && anyPump())
  {
    trip("mixing tank full (float FSW5)");
    return;
  }
  if (!doseActive())
  {
    for (int p = 0; p < 3; p++)
    {
      if (!pumpPct[p])
        continue;
      int s = p;
      uint32_t since = max(pumpOnMs[p], lastCountChangeMs[s]);
      uint32_t lim = lastCountChangeMs[s] > pumpOnMs[p] ? STALL_MS : START_STALL_MS;
      if (now - since > lim)
      {
        char why[48];
        snprintf(why, sizeof(why), "no flow on P%d (manual run)", p + 1);
        trip(why);
        return;
      }
    }
  }
}

static uint8_t dtConfirmN = 0;
static uint32_t dtT0 = 0, lastTempSeen = 0, pauseT0 = 0;
static float loopRefT = NAN;
static uint32_t loopRefMs = 0;
static uint16_t loopChecks = 0;
static int doseIdx = 0;

static void logStep(const char *what) { Serial.printf("S,%.1f,%s\n", runSecs(), what); }

static void recordResult(int line)
{
  float hold = (line == L_RED && seq == S_DOSE) ? cfg.redHoldupMl : 0;
  res[line] = {true, dose.target - hold, dose.mlFinal - hold, (millis() - dose.t0) / 1000.0f, dose.p
  if (hold > 0)
    Serial.printf("H,%d,%s,%.1f,%.1f\n", line + 1, LD[line].name, hold, dose.mlFinal);
  Serial.printf("R,%d,%s,%.1f,%.1f,%+.1f,%lu,%.1f\n", line + 1, LD[line].name, res[line].target, res
                res[line].ml - res[line].target, (unsigned long)dose.pulsesFinal, res[line].secs);
}

static bool startDoseIdx()
{
```

```
  int l = cfg.order[doseIdx];
  float extra = l == L_RED ? cfg.redHoldupMl : 0;
  if (extra > 0)
    Serial.printf("> RED line still holds ~%.1f mL of flush water from Step 1, metering %.1f mL so %
                  extra, cfg.line[l].target + extra, cfg.line[l].target);
  return doseBegin(l, cfg.line[l].target + extra, false, false, 0, 0);
}

static void finishRun()
{
  allOff();
  seq = S_DONE;
  logStep("DONE");
  float tot = 0;
  for (int i = 0; i < 3; i++)
    tot += res[i].ml;
  Serial.println(F("\n===== SEQUENCE COMPLETE ====="));
  Serial.printf("Step 1 flush: %.1f mL of %.1f mL to the jug in %.1f s\n", res[L_WATER].ml, res[L_WA
  Serial.printf("Step 2 dT: %.2f C reached after %.1f s\n", dtReached, dtSecs);
  for (int i = 0; i < 3; i++)
  {
    int l = cfg.order[i];
    Serial.printf("Step 3 %-7s target %7.1f mL  metered %7.1f mL  error %+6.1f mL (%+.2f %%)  ratio
                  res[l].target, res[l].ml, res[l].ml - res[l].target, (res[l].ml - res[l].target) *
                  tot > 0 ? res[l].ml * 100.0f / tot : 0);
  }
  Serial.printf("Total colour %.1f mL, run time %.1f s\n", tot, runSecs());
  Serial.print(F("SAFE STATE: "));
  for (int p = 0; p < 4; p++)
    Serial.printf("P%d=%u%% ", p + 1, pumpPct[p]);
  for (int v = 0; v < NSV; v++)
    Serial.printf("SV%d=%s ", v + 1, valveOpen[v] ? "OPEN" : "closed");
  Serial.println();
  Serial.println(F("=============================="));
  logTemp(true);
  runLogging = false;
  printReport();
  Serial.println(F(">>> All pumps OFF, all valves CLOSED. Press START for a new run (empty the jug f
}

static void enterDose()
{
  if (pumpPct[P_LOOP] || valveOpen[SV_LOOP])
    loopStop();
  seq = S_DOSE;
  logStep("STEP3_START");
  doseIdx = 0;
  if (!startDoseIdx())
    trip("first colour dose would not start");
}

static void enterDt()
{
  seq = S_DT;
```

```
  dtT0 = millis();
  dtConfirmN = 0;
  lastTempSeen = tempFrames;
  loopRefT = tempC[0];
  loopRefMs = dtT0;
  loopChecks = 0;
  logStep("STEP2_START");
  if (cfg.dtTarget <= 0)
  {
    Serial.println(F("> STEP 2 skipped (dt set to 0, test mode)"));
    logStep("STEP2_SKIPPED");
    enterDose();
    return;
  }
  if (tempsOk() && deltaT() >= cfg.dtTarget + cfg.dtMargin)
    Serial.printf("> STEP 2: dT is already %.2f C, confirming over %u readings\n", deltaT(), cfg.dtC
  if (!loopStart())
  {
    trip("cooling loop would not start");
    return;
  }
  Serial.printf("> STEP 2: SV5 open, P4 %u%% circulating YELLOW through the ice-bath coil until |TS1
                cfg.loopDuty, cfg.dtTarget + cfg.dtMargin);
}

static void seqStep()
{
  uint32_t now = millis();
  switch (seq)
  {
  case S_FLUSH:
    if (dose.ph == D_FAULT)
      break;
    if (dose.ph == D_DONE)
    {
      recordResult(L_WATER);
      logStep("STEP1_DONE");
      Serial.printf("> STEP 1 DONE: %.1f mL flushed to the jug, P1 off, SV4 closed\n", dose.mlFinal)
      Serial.println(F(">>> LIFT THE JUG OUT OF THE MIXING TANK NOW <<<"));
      if (cfg.pauseS)
      {
        seq = S_PAUSE;
        pauseT0 = now;
        logStep("PAUSE_START");
        Serial.printf("> pause %lu s, Step 2 starts by itself\n", (unsigned long)cfg.pauseS);
      }
      else
        enterDt();
    }
    break;
  case S_PAUSE:
    if (now - pauseT0 >= cfg.pauseS * 1000UL)
    {
      logStep("PAUSE_DONE");
```

```
      enterDt();
    }
    break;
  case S_DT:
  {
    if (!tempsOk())
    {
      if (now - dtT0 > 5000)
        trip("temperature sensor missing or not reporting");
      break;
    }
    if (isnan(loopRefT))
      loopRefT = tempC[0];
    if (cfg.loopCheckS && now - loopRefMs > cfg.loopCheckS * 1000UL)
    {
      float drop = loopRefT - tempC[0];
      Serial.printf("~ cooling check %u: TS1 dropped %.2f C in the last %lu s (need %.2f)\n", ++loop
                    (unsigned long)cfg.loopCheckS, cfg.loopMinDrop);
      if (drop < cfg.loopMinDrop && deltaT() < cfg.dtTarget + cfg.dtMargin)
      {
        trip(loopChecks == 1 ? "no circulation in the cooling loop (TS1 not dropping)" : "cooling st
        break;
      }
      loopRefT = tempC[0];
      loopRefMs = now;
    }
    if (tempFrames != lastTempSeen && lastValidMs[0] == lastTempMs && lastValidMs[1] == lastTempMs)
    {
      lastTempSeen = tempFrames;
      if (deltaT() >= cfg.dtTarget + cfg.dtMargin)
        dtConfirmN++;
      else
        dtConfirmN = 0;
      if (dtConfirmN >= cfg.dtConfirm)
      {
        dtReached = deltaT();
        dtSecs = (now - dtT0) / 1000.0f;
        loopStop();
        logTemp(true);
        Serial.printf("> STEP 2 DONE: dT = %.2f C (TS1 %.2f, TS2 %.2f) after %.1f s, P4 off, SV5 clo
        logStep("STEP2_DONE");
        enterDose();
        break;
      }
    }
    if (cfg.dtTimeoutMin && now - dtT0 > cfg.dtTimeoutMin * 60000UL)
      trip("dT not reached in time");
    break;
  }
  case S_DOSE:
    if (dose.ph == D_FAULT)
      break;
    if (dose.ph == D_DONE)
    {
```

```
      recordResult(dose.line);
      if (++doseIdx < 3)
      {
        if (!startDoseIdx())
          trip("next colour dose would not start");
      }
      else
      {
        logStep("STEP3_DONE");
        finishRun();
      }
    }
    break;
  default:
    break;
  }
}

static bool running() { return seq == S_FLUSH || seq == S_PAUSE || seq == S_DT || seq == S_DOSE || d

static void runStart()
{
  if (running())
  {
    Serial.println(F("! busy"));
    return;
  }
  for (int i = 0; i < NLINES; i++)
    if (cfg.line[i].k <= 0)
    {
      Serial.printf("! %s not calibrated (k=0), run refused\n", LD[i].name);
      return;
    }
  if (!linkOk())
  {
    Serial.println(F("! no data from sensor board U7, run refused"));
    return;
  }
  if (cfg.dtTarget > 0 && !tempsOk())
  {
    Serial.println(F("! temperature sensors TS1/TS2 not both reading, run refused (type temps)"));
    return;
  }
  float need = cfg.line[0].target + cfg.line[1].target + cfg.line[2].target + cfg.redHoldupMl;
  if (mixMl + need > cfg.mixMaxMl)
  {
    Serial.printf("! mixing tank count is %.0f mL, + %.0f mL would pass %.0f mL. Empty it and type m
    return;
  }
  allOff();
  jugMl = 0;
  nT = nD = 0;
  tStride = 1;
  tTick = 0;
```

```
  for (int i = 0; i < NLINES; i++)
    res[i] = Result();
  dtReached = NAN;
  dtSecs = 0;
  tripWhy[0] = 0;
  runNo++;
  runT0 = millis();
  runLogging = true;
  Serial.printf("\n#RUN,%lu\n", (unsigned long)runNo);
  Serial.printf(">>> RUN %lu START. Step 1 flush %.0f mL water -> jug, Step 2 dT %.1f C by cooling,
                (unsigned long)runNo, cfg.line[L_WATER].target, cfg.dtTarget,
                LD[cfg.order[0]].name, cfg.line[cfg.order[0]].target, LD[cfg.order[1]].name, cfg.lin
                LD[cfg.order[2]].name, cfg.line[cfg.order[2]].target);
  logStep("STEP1_START");
  logTemp(true);
  seq = S_FLUSH;
  if (!doseBegin(L_WATER, cfg.line[L_WATER].target, true, false, 0, 0))
    trip("flush would not start");
}

static bool rawAverage(float out[2], float spread[2])
{
  if (nHist < TAVG)
    return false;
  for (int p = 0; p < 2; p++)
  {
    float sum = 0, lo = 1e9f, hi = -1e9f;
    for (int i = 0; i < TAVG; i++)
    {
      float v = rawHist[p][i];
      if (isnan(v))
        return false;
      sum += v;
      lo = min(lo, v);
      hi = max(hi, v);
    }
    out[p] = sum / TAVG;
    spread[p] = hi - lo;
  }
  return true;
}

static void addQ(int probe, char method, float raw, float ref, float off)
{
  if (nQ < 8)
    qlog[nQ++] = {(uint8_t)probe, method, raw, ref, off};
  Serial.printf("Q,%d,TS%d,%s,%.3f,%.3f,%+.3f\n", probe + 1, probe + 1, method == 'i' ? "ice-point"
}

static void tempCal(const char *how)
{
  float avg[2], spread[2];
  if (!rawAverage(avg, spread))
  {
```

```
    Serial.println(F("! need 8 good readings from both probes first (about 7 s), try again"));
    return;
  }
  if (spread[0] > 0.25f || spread[1] > 0.25f)
  {
    Serial.printf("! readings still moving (TS1 spread %.2f C, TS2 %.2f C). Stir, wait 1 minute, try
    return;
  }
  String h(how);
  if (h == "ice")
  {
    if (fabsf(avg[0]) > 2.0f || fabsf(avg[1]) > 2.0f)
    {
      Serial.printf("! probes read %.2f / %.2f C, not near 0 C. Put both in stirred ice water (more
      return;
    }
    for (int p = 0; p < 2; p++)
    {
      cfg.tsOffset[p] = -avg[p];
      addQ(p, 'i', avg[p], 0.0f, cfg.tsOffset[p]);
    }
  }
  else if (h == "match")
  {
    float target = avg[0] + cfg.tsOffset[0];
    cfg.tsOffset[1] = target - avg[1];
    addQ(1, 'm', avg[1], target, cfg.tsOffset[1]);
  }
  else if (h == "clear")
  {
    cfg.tsOffset[0] = cfg.tsOffset[1] = 0;
    Serial.println(F("probe offsets cleared"));
    return;
  }
  else
  {
    Serial.println(F("usage: tcal ice | tcal match | tcal clear"));
    return;
  }
  Serial.printf("offsets now TS1 %+.2f C, TS2 %+.2f C. Type code and paste TUNE_TS_OFFSET to keep th
}

static void printCal()
{
  Serial.println(F("K,line,name,k_pulses_per_mL,sensor"));
  for (int i = 0; i < NLINES; i++)
    Serial.printf("K,%d,%s,%.4f,FT%d\n", i + 1, LD[i].name, cfg.line[i].k, sensorOf(i) + 1);
  for (int i = 0; i < nC; i++)
    Serial.printf("C,%d,%s,%lu,%lu,%.1f,%.4f\n", clog[i].line + 1, LD[clog[i].line].name, (unsigned
                  (unsigned long)clog[i].ms, clog[i].ml, clog[i].k);
  Serial.printf("O,TS1,%+.3f\nO,TS2,%+.3f\n", cfg.tsOffset[0], cfg.tsOffset[1]);
  for (int i = 0; i < nQ; i++)
    Serial.printf("Q,%d,TS%d,%s,%.3f,%.3f,%+.3f\n", qlog[i].probe + 1, qlog[i].probe + 1, qlog[i].me
                  qlog[i].raw, qlog[i].ref, qlog[i].offset);
```

```
}

static void printReport()
{
  Serial.printf("#REPORT BEGIN,%lu\n", (unsigned long)runNo);
  Serial.printf("P,dt_target,%.1f\nP,mode,cooling\n", cfg.dtTarget);
  for (int i = 0; i < NLINES; i++)
    Serial.printf("P,target%d,%.1f\n", i + 1, cfg.line[i].target);
  printCal();
  for (int i = 0; i < NLINES; i++)
    if (res[i].done)
      Serial.printf("R,%d,%s,%.1f,%.1f,%+.1f,%lu,%.1f\n", i + 1, LD[i].name, res[i].target, res[i].m
                    (unsigned long)res[i].pulses, res[i].secs);
  if (!isnan(dtReached))
    Serial.printf("X,dt_reached,%.2f,%.1f\n", dtReached, dtSecs);
  if (tripWhy[0])
    Serial.printf("X,trip,%s\n", tripWhy);
  for (int i = 0; i < nT; i++)
    Serial.printf("T,%.1f,%.2f,%.2f,%.2f,%u\n", tlog[i].t, tlog[i].a, tlog[i].b, fabsf(tlog[i].a - t
  for (int i = 0; i < nD; i++)
    Serial.printf("D,%.2f,%u,log,%u,0,%.1f,%.1f\n", dlog[i].t, dlog[i].line, dlog[i].duty, dlog[i].m
  Serial.println(F("#REPORT END"));
}

static void pollStart()
{
  static bool last = true, stable = true;
  static uint32_t tChange = 0;
  bool r = digitalRead(START_PIN);
  if (r != last)
  {
    last = r;
    tChange = millis();
  }
  if (millis() - tChange > 40 && r != stable)
  {
    stable = r;
    if (!stable)
    {
      if (running() && millis() - runT0 < TUNE_START_LOCKOUT_MS)
        Serial.println(F("~ START ignored (pressed again right after the start)"));
      else if (running())
        trip("START pressed during run (used as stop)");
      else
        runStart();
    }
  }
}

static void printConfig()
{
  Serial.println(F("line      k(p/mL)  target  fast%  slow%  overrun  route"));
  static const char *ROUTE[] = {"SV1>TA>P1>FT1>mix", "TC>P2>FT2>SV2>mix", "P3>FT3>SV3>mix", "SV4>TA>
  for (int i = 0; i < NLINES; i++)
```

```
    Serial.printf("%d %-6s %8.4f %7.1f %5u %6u %8.1f  %s\n", i + 1, LD[i].name, cfg.line[i].k, cfg.l
                  cfg.line[i].fast, cfg.line[i].slow, cfg.line[i].overrun, ROUTE[i]);
  Serial.printf("order %d%d%d  band %.1f %% (min %.0f mL)  learn %d  red hold-up %.1f mL\n", cfg.ord
                cfg.order[2] + 1, cfg.bandPct, cfg.bandMinMl, cfg.learn, cfg.redHoldupMl);
  Serial.printf("pause after Step 1 %lu s  ", (unsigned long)cfg.pauseS);
  Serial.printf("dT target %.1f C (+%.1f margin, %u readings)  timeout %lu min  loop P4 %u%%  check
                cfg.dtTarget, cfg.dtMargin, cfg.dtConfirm, (unsigned long)cfg.dtTimeoutMin, cfg.loop
                (unsigned long)cfg.loopCheckS);
  Serial.printf("jug ~%.0f mL (max %.0f)  mixing tank ~%.0f mL (max %.0f)\n", jugMl, cfg.jugMaxMl, m
}
static void printTemps()
{
  if (tempFrames == 0)
  {
    Serial.println(F("temps: nothing from U7 yet"));
    return;
  }
  Serial.printf("TS1 (yellow, cooled) %.2f C  TS2 (reference) %.2f C  dT %.2f C  loop P4 %s  age %lu
                deltaT(), pumpPct[P_LOOP] ? "ON" : "off", (unsigned long)(millis() - lastTempMs));
  Serial.printf("  raw %.2f / %.2f C, offsets %+.2f / %+.2f C, bad readings so far %lu\n", tempRaw[0
                cfg.tsOffset[1], (unsigned long)badTempFrames);
}
static void printStatus()
{
  Serial.printf("link %s (ok %lu, bad %lu)  counts %lu %lu %lu  start=%s\n", linkOk() ? "OK" : "DOWN
                (unsigned long)framesOk, (unsigned long)framesBad, (unsigned long)counts[0], (unsign
                (unsigned long)counts[2], digitalRead(START_PIN) ? "up" : "PRESSED");
  Serial.print(F("valves "));
  for (int v = 0; v < NSV; v++)
    Serial.printf("SV%d=%d ", v + 1, valveOpen[v]);
  Serial.print(F(" pumps "));
  for (int p = 0; p < 4; p++)
    Serial.printf("P%d=%u%% ", p + 1, pumpPct[p]);
  Serial.printf("\nfloats FSW1-4 %d%d%d%d FSW5 %d FSW6 %d\n", digitalRead(FSW_PIN[0]), digitalRead(F
                digitalRead(FSW_PIN[2]), digitalRead(FSW_PIN[3]), fswU7 & 1, (fswU7 >> 1) & 1);
  printTemps();
  Serial.printf("sequence %s  dose %s%s%s\n", SEQ_NAME[seq], PHASE_NAME[dose.ph], tripWhy[0] ? "  la
}
static void help()
{
  Serial.println(F(
      "--- Team 13 Chocolate Day 2 controller, ESP32 #1 (U2) ---\n"
      "lines: 1=RED 2=YELLOW 3=GREEN 4=WATER (water runs SV4 > TA > P1 > FT1 > jug in the mixing tan
      "valves: SV1 red source  SV2 yellow  SV3 green  SV4 water source  SV5 cooling loop\n"
      "start | abort          run Step 1 flush > Step 2 dT (cooling) > Step 3 dose (same as START sw
      "status | show | temps  live I/O | settings | temperatures\n"
      "prime <1-4> <sec>      fill a line at fast duty (1 and 4 counted as jug)\n"
      "calrun <1-4> <sec> [%] timed run, counts pulses\n"
      "k <1-4> <mL>           set k = pulses of last calrun/dose / mL weighed (adds a calibration ro
      "cal                    print the calibration table (flow k and probe offsets)\n"
      "tcal ice | match | clear  probe calibration: both probes in stirred ice water | in the same w
      "dose <1-4> <mL> [jug]  metered dose (default counted into the mixing tank)\n"
      "loop <0|1>             cooling loop P4 + SV5 by hand, idle only\n"
      "tlog <0|1>             log temperatures every 2 s while idle\n"
```

```
      "report                 print the stored data of the last run (flush, cal, dT trend, doses)\n"
      "jug <mL> | mix <mL>    set the jug / mixing tank volume count (0 after emptying)\n"
      "valve <1-5> <0|1>      manual valve, idle only\n"
      "pump <1-4> <%>         manual pump, idle only, interlocked\n"
      "off                    everything off\n"
      "set <key> <value>      k1..4 tgt1..4 fast1..4 slow1..4 ovr1..4 order(e.g. 231) band bandmin l
      "                       dt dtmargin dtconfirm dttimeout loopduty loopcheck loopdrop mixmax jug
      "save | load | defaults | code"));
}

static void handleCommand(String line)
{
  line.trim();
  if (!line.length())
    return;
  char buf[96];
  line.toCharArray(buf, sizeof(buf));
  char *cmd = strtok(buf, " ");
  char *a1 = strtok(nullptr, " ");
  char *a2 = strtok(nullptr, " ");
  char *a3 = strtok(nullptr, " ");
  String c(cmd);
  c.toLowerCase();
  int n = a1 ? atoi(a1) : 0;
  bool busy = running();

  if (c == "help" || c == "?")
    help();
  else if (c == "status")
    printStatus();
  else if (c == "show")
    printConfig();
  else if (c == "temps")
    printTemps();
  else if (c == "cal")
    printCal();
  else if (c == "report")
    printReport();
  else if (c == "tcal" && !busy)
    tempCal(a1 ? a1 : "");
  else if (c == "start")
    runStart();
  else if (c == "abort" || c == "stop")
    trip("abort command");
  else if (c == "off")
  {
    allOff();
    dose.ph = D_IDLE;
    seq = S_IDLE;
    runLogging = false;
    Serial.println(F("all off"));
  }
  else if (c == "tlog" && a1)
  {
```

```
  tlogIdle = n != 0;
  Serial.printf("idle temperature log %s\n", tlogIdle ? "on" : "off");
}
else if (busy)
  Serial.println(F("! busy, type abort first"));
else if (c == "prime" && n >= 1 && n <= 4 && a2)
  doseBegin(n - 1, 0, n == 1 || n == 4, true, (uint32_t)(atof(a2) * 1000), cfg.line[n - 1].fast);
else if (c == "calrun" && n >= 1 && n <= 4 && a2)
{
  uint8_t duty = a3 ? atoi(a3) : cfg.line[n - 1].fast;
  doseBegin(n - 1, 0, n == 1 || n == 4, true, (uint32_t)(atof(a2) * 1000), duty);
}
else if (c == "k" && n >= 1 && n <= 4 && a2)
{
  float ml = atof(a2);
  if (ml <= 0 || lastPulses[n - 1] == 0)
    Serial.println(F("! need a calrun or dose on that line first, and mL > 0"));
  else
  {
    cfg.line[n - 1].k = lastPulses[n - 1] / ml;
    if (nC < MAX_C)
      clog[nC++] = {(uint8_t)(n - 1), lastPulses[n - 1], lastRunMs[n - 1], ml, cfg.line[n - 1].k};
    Serial.printf("C,%d,%s,%lu,%lu,%.1f,%.4f\n", n, LD[n - 1].name, (unsigned long)lastPulses[n -
                  (unsigned long)lastRunMs[n - 1], ml, cfg.line[n - 1].k);
    Serial.printf("k%d = %lu / %.1f = %.4f pulses/mL (type code and paste TUNE_K to keep it)\n", n
                  (unsigned long)lastPulses[n - 1], ml, cfg.line[n - 1].k);
  }
}
else if (c == "dose" && n >= 1 && n <= 4 && a2)
{
  bool jug = a3 && String(a3) == "jug";
  doseBegin(n - 1, atof(a2), jug, false, 0, 0);
}
else if (c == "loop" && a1)
{
  if (n)
  {
    if (loopStart())
      Serial.println(F("cooling loop ON (SV5 open, P4 running). No flow sensor on this loop: watch
  }
  else
  {
    loopStop();
    Serial.println(F("cooling loop off"));
  }
}
else if (c == "jug" && a1)
{
  jugMl = atof(a1);
  Serial.printf("jug ~%.0f mL\n", jugMl);
}
else if (c == "mix" && a1)
{
  mixMl = atof(a1);
```

```
  Serial.printf("mixing tank ~%.0f mL\n", mixMl);
}
else if (c == "valve" && n >= 1 && n <= NSV && a2)
{
  setValve(n - 1, atoi(a2));
  printStatus();
}
else if (c == "pump" && n >= 1 && n <= 4 && a2)
{
  setPump(n - 1, atoi(a2));
  printStatus();
}
else if (c == "save")
  saveConfig();
else if (c == "code")
  printCode();
else if (c == "load")
{
  loadConfig();
  printConfig();
}
else if (c == "defaults")
{
  setDefaults();
  printConfig();
  Serial.println(F("(not saved yet)"));
}
else if (c == "set" && a1 && a2)
{
  String k(a1);
  k.toLowerCase();
  float v = atof(a2);
  int i = k.length() > 0 ? k.charAt(k.length() - 1) - '1' : -1;
  String base = k.substring(0, k.length() - 1);
  bool ok = true;
  if (i >= 0 && i < NLINES && base == "k")
    cfg.line[i].k = v;
  else if (i >= 0 && i < NLINES && base == "tgt")
    cfg.line[i].target = v;
  else if (i >= 0 && i < NLINES && base == "fast")
    cfg.line[i].fast = constrain((int)v, 0, 100);
  else if (i >= 0 && i < NLINES && base == "slow")
    cfg.line[i].slow = constrain((int)v, 0, 100);
  else if (i >= 0 && i < NLINES && base == "ovr")
    cfg.line[i].overrun = v;
  else if (k == "order" && strlen(a2) == 3)
  {
    uint8_t o[3];
    bool seen[3] = {false, false, false};
    for (int j = 0; j < 3; j++)
    {
      o[j] = a2[j] - '1';
      if (o[j] > 2 || seen[o[j]])
        ok = false;
```

```
        else
          seen[o[j]] = true;
      }
      if (ok)
        memcpy(cfg.order, o, 3);
    }
    else if (k == "band")
      cfg.bandPct = v;
    else if (k == "bandmin")
      cfg.bandMinMl = v;
    else if (k == "learn")
      cfg.learn = v != 0;
    else if (k == "dt")
      cfg.dtTarget = v;
    else if (k == "dtmargin")
      cfg.dtMargin = v;
    else if (k == "dtconfirm")
      cfg.dtConfirm = constrain((int)v, 1, 50);
    else if (k == "dttimeout")
      cfg.dtTimeoutMin = (uint32_t)v;
    else if (k == "loopduty")
      cfg.loopDuty = constrain((int)v, 0, 100);
    else if (k == "loopcheck")
      cfg.loopCheckS = (uint32_t)v;
    else if (k == "loopdrop")
      cfg.loopMinDrop = v;
    else if (k == "mixmax")
      cfg.mixMaxMl = v;
    else if (k == "jugmax")
      cfg.jugMaxMl = v;
    else if (k == "ts1off")
      cfg.tsOffset[0] = v;
    else if (k == "ts2off")
      cfg.tsOffset[1] = v;
    else if (k == "pause")
      cfg.pauseS = constrain((int)v, 0, 3600);
    else if (k == "holdup")
      cfg.redHoldupMl = constrain(v, 0.0f, 200.0f);
    else
      ok = false;
    if (ok)
    {
      printConfig();
      Serial.println(F("(type code to get the TUNING block)"));
    }
    else
      Serial.println(F("! unknown key or bad value"));
  }
  else
    Serial.println(F("? type help"));
}

void setup()
{
```

```
  for (int v = 0; v < NSV; v++)
  {
    pinMode(SV_PIN[v], OUTPUT);
    digitalWrite(SV_PIN[v], LOW);
  }
  for (int p = 0; p < 4; p++)
  {
    ledcSetup(p, PWM_FREQ, PWM_BITS);
    ledcAttachPin(PWM_PIN[p], p);
    ledcWrite(p, 0);
  }
  for (int i = 0; i < 4; i++)
    pinMode(FSW_PIN[i], INPUT);
  pinMode(START_PIN, INPUT_PULLUP);
  Serial.begin(115200);
  Serial2.setRxBufferSize(1024);
  Serial2.begin(LINK_BAUD, SERIAL_8N1, LINK_RX_PIN, LINK_TX_PIN);
  loadConfig();
  delay(300);
  Serial.println(F("\n[U2 controller] Team 13 Chocolate, Day 2: Flush > dT 10 C (cooling) > Dose. Ty
  printConfig();
}

void loop()
{
  pollLink();
  pollStart();
  interlocks();
  doseStep();
  seqStep();
  logTemp(false);

  static String rx;
  while (Serial.available())
  {
    char ch = Serial.read();
    if (ch == '\n' || ch == '\r')
    {
      if (rx.length())
      {
        pollLink();
        handleCommand(rx);
        rx = "";
      }
    }
    else if (rx.length() < 90)
    {
      rx += ch;
    }
  }
}
```